

Predicting Stock Market Direction Using Classification Models

Manuela Lozano

2026-04-23

Contents

1	Abstract	2
2	Introduction	2
2.1	Background Information	2
2.2	Research Question	2
2.3	Hypothesis	2
3	Data Description	2
4	Exploratory Data Analysis	4
5	Methodology	10
5.1	Training Strategy	10
5.2	Logistic Regression	11
5.3	LDA	12
5.4	QDA	14
5.5	kNN	16
5.6	Cross Validation for ALL Models	17
6	Results	21
6.1	Accuracy Comparison	21
6.2	Training vs Testing Accuracy	21
6.3	Confusion Matrices	23
6.4	ROC Curves and AUC	25
6.5	Linear vs. Nonlinear Discussion	27
7	Conclusion and Future Work	28
7.1	Economic Interpretation	28
7.2	Final Thoughts	28
7.3	Future Work	28
8	References	28

1 Abstract

This study evaluates the ability of several classification models to predict daily stock market direction using lagged financial indicators. Using S&P 500 data from 2010 to 2024, logistic regression, Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and k-Nearest Neighbors (kNN) are implemented and compared. Model performance is assessed using out-of-sample accuracy, confusion matrices, cross-validation, and the Area Under the ROC Curve (AUC). The results show that all models perform close to random guessing, indicating limited predictive power. While kNN slightly outperforms the other models, the improvement is minimal, suggesting that any nonlinear patterns in the data are weak. Overall, the findings are consistent with the weak-form Efficient Market Hypothesis, indicating that past returns contain little useful information for predicting future market direction.

2 Introduction

2.1 Background Information

Predicting stock market movements has long been a central challenge in financial economics, as it directly impacts investment decisions, portfolio allocation, and risk management. According to the weak-form Efficient Market Hypothesis (EMH), asset prices fully incorporate all past price information, implying that historical returns should not provide reliable signals for forecasting future market direction. Even in cases where small predictive patterns may exist, their economic value is often limited once transaction costs, market friction, and risk considerations are taken into account.

At the same time, classification methods from statistical learning provide a structured framework for evaluating whether predictive relationships exist in financial data. Techniques such as logistic regression, Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and k-Nearest Neighbors (kNN) are commonly used to model categorical outcomes and assess whether past information can be used to predict future events.

2.2 Research Question

To what extent can lagged financial indicators predict daily stock market direction, and do nonlinear classification methods provide meaningful improvements over linear models?

2.3 Hypothesis

We hypothesize that predictive performance will be limited, and that nonlinear models will not significantly outperform linear models, consistent with the Efficient Market Hypothesis.

3 Data Description

The data used in this study consists of daily S&P 500 index prices obtained from Yahoo Finance using the *quantmod* package in R. The sample period spans from January 2010 to January 2024, providing a total of 3,522 observations. Daily returns are computed from adjusted closing prices and serve as the primary variable of interest.

A binary response variable, *Direction*, is constructed to indicate whether the market increased (“Up”) or decreased (“Down”) on a given day. To capture potential temporal dependence, lagged returns from the previous five trading days (*Lag1-Lag5*) are created and used as predictors. These lagged variables ensure that only past information is used for prediction, avoiding look-ahead bias.

```
# Download S&P 500 data from Yahoo Finance
getSymbols("^GSPC", from = "2010-01-01", to = "2024-01-01")
```

```
## [1] "GSPC"
```

```
# Compute daily returns from closing prices
returns <- dailyReturn(Cl(GSPC))
```

```
# Convert to data frame
data <- data.frame(
  Date = index(returns),
  Return = as.numeric(returns)
)
```

```
# Create binary response variable
data$Direction <- ifelse(data$Return > 0, "Up", "Down")
```

```
# Make Direction a factor
data$Direction <- as.factor(data$Direction)
```

```
head(data)
```

```
##           Date      Return Direction
## 1 2010-01-04 0.0000000000      Down
## 2 2010-01-05 0.0031156756       Up
## 3 2010-01-06 0.0005455206       Up
## 4 2010-01-07 0.0040012019       Up
## 5 2010-01-08 0.0028817273       Up
## 6 2010-01-11 0.0017467554       Up
```

```
str(data)
```

```
## 'data.frame': 3522 obs. of 3 variables:
## $ Date : Date, format: "2010-01-04" "2010-01-05" ...
## $ Return : num 0 0.003116 0.000546 0.004001 0.002882 ...
## $ Direction: Factor w/ 2 levels "Down","Up": 1 2 2 2 2 2 1 2 2 1 ...
```

```
# Create Lagged Predictors
```

```
data <- data %>%
  mutate(
    Lag1 = lag(Return, 1),
    Lag2 = lag(Return, 2),
    Lag3 = lag(Return, 3),
    Lag4 = lag(Return, 4),
    Lag5 = lag(Return, 5)
  ) %>%
  na.omit()
```

```
head(data)
```

```
##           Date      Return Direction      Lag1      Lag2      Lag3
## 6 2010-01-11 0.001746755      Up 0.002881727 0.004001202 0.0005455206
## 7 2010-01-12 -0.009381166     Down 0.001746755 0.002881727 0.0040012019
## 8 2010-01-13 0.008325926      Up -0.009381166 0.001746755 0.0028817273
## 9 2010-01-14 0.002426425      Up 0.008325926 -0.009381166 0.0017467554
## 10 2010-01-15 -0.010823130     Down 0.002426425 0.008325926 -0.0093811661
```

```
## 11 2010-01-19 0.012499627 Up -0.010823130 0.002426425 0.0083259257
##           Lag4           Lag5
## 6 0.0031156756 0.0000000000
## 7 0.0005455206 0.0031156756
## 8 0.0040012019 0.0005455206
## 9 0.0028817273 0.0040012019
## 10 0.0017467554 0.0028817273
## 11 -0.0093811661 0.0017467554
```

```
summary(data)
```

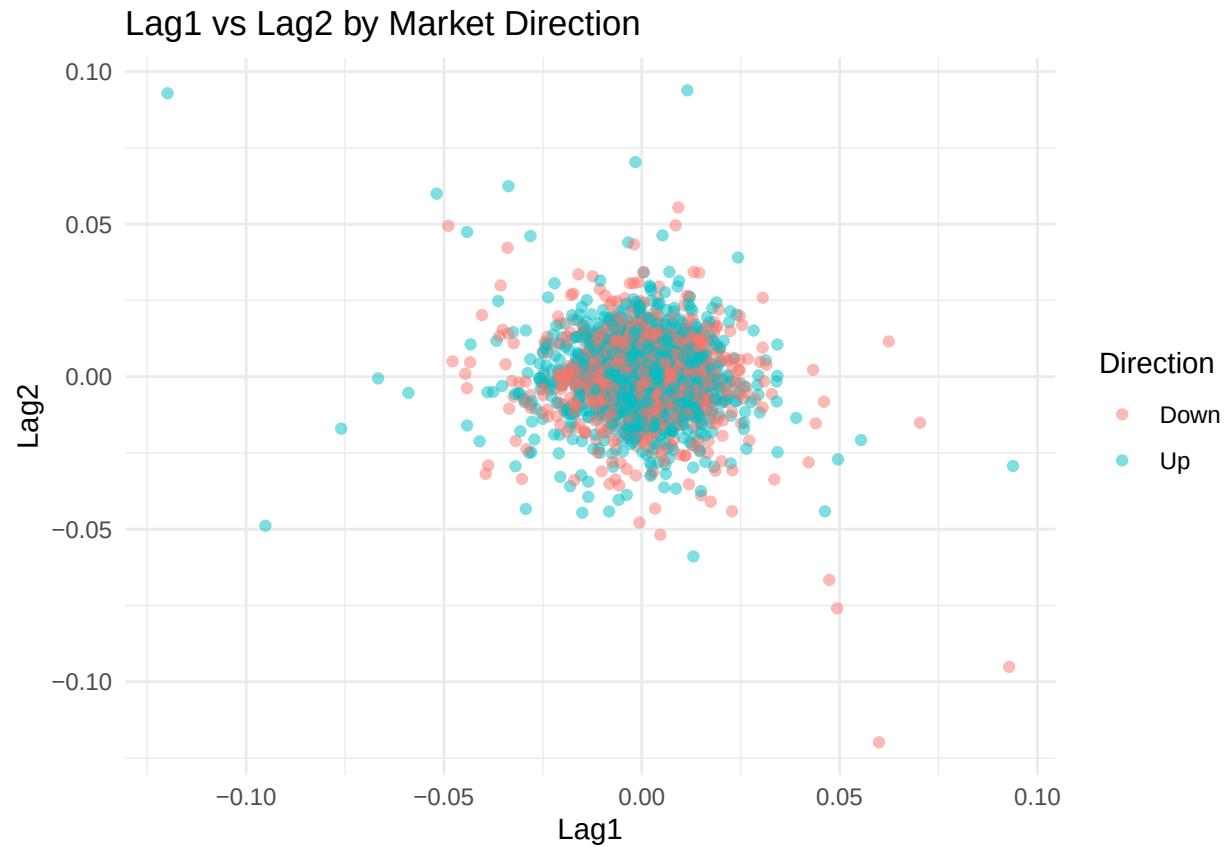
```
##           Date           Return           Direction           Lag1
## Min.      :2010-01-11   Min.      :-0.1198406   Down:1609   Min.      :-0.1198406
## 1st Qu.:2013-07-10   1st Qu.: -0.0038837   Up  :1908   1st Qu.: -0.0038837
## Median :2017-01-04   Median : 0.0006359                Median : 0.0006428
## Mean    :2017-01-03   Mean    : 0.0004670                Mean    : 0.0004686
## 3rd Qu.:2020-07-02   3rd Qu.: 0.0056979                3rd Qu.: 0.0056979
## Max.    :2023-12-29   Max.    : 0.0938277                Max.    : 0.0938277
##           Lag2           Lag3           Lag4
## Min.      :-0.1198406   Min.      :-0.1198406   Min.      :-0.1198406
## 1st Qu.: -0.0038837   1st Qu.: -0.0038837   1st Qu.: -0.0038837
## Median : 0.0006478   Median : 0.0006428   Median : 0.0006428
## Mean     : 0.0004696   Mean     : 0.0004694   Mean     : 0.0004691
## 3rd Qu.: 0.0056979   3rd Qu.: 0.0056979   3rd Qu.: 0.0056979
## Max.     : 0.0938277   Max.     : 0.0938277   Max.     : 0.0938277
##           Lag5
## Min.      :-0.1198406
## 1st Qu.: -0.0038837
## Median : 0.0006359
## Mean     : 0.0004686
## 3rd Qu.: 0.0056979
## Max.     : 0.0938277
```

Interpretation:

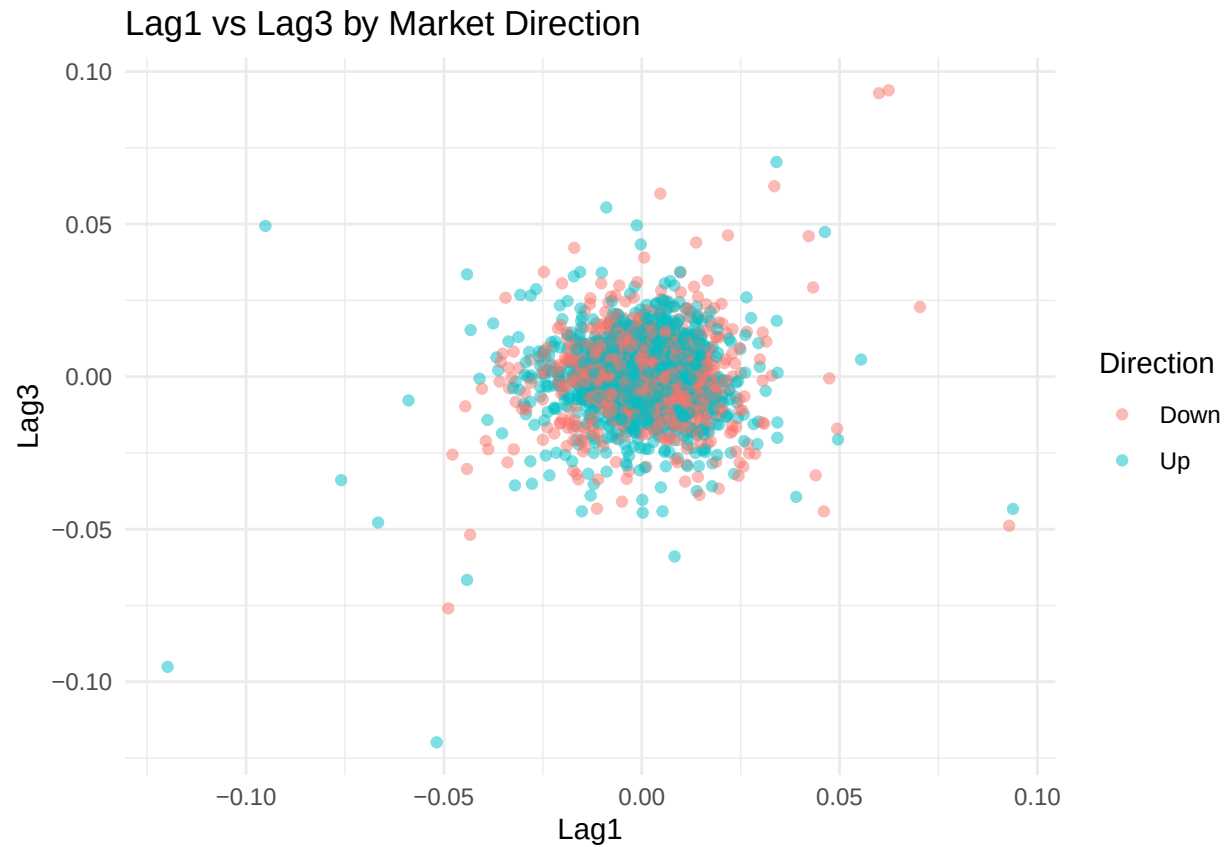
The summary statistics show that daily returns are centered close to zero, with small average values and occasional extreme movements. This is consistent with typical financial return data, where most daily changes are minor but large shocks can occur. The Direction variable is relatively balanced between “Up” and “Down”, which is important for classification because it prevents accuracy from being driven by severe class imbalance.

4 Exploratory Data Analysis

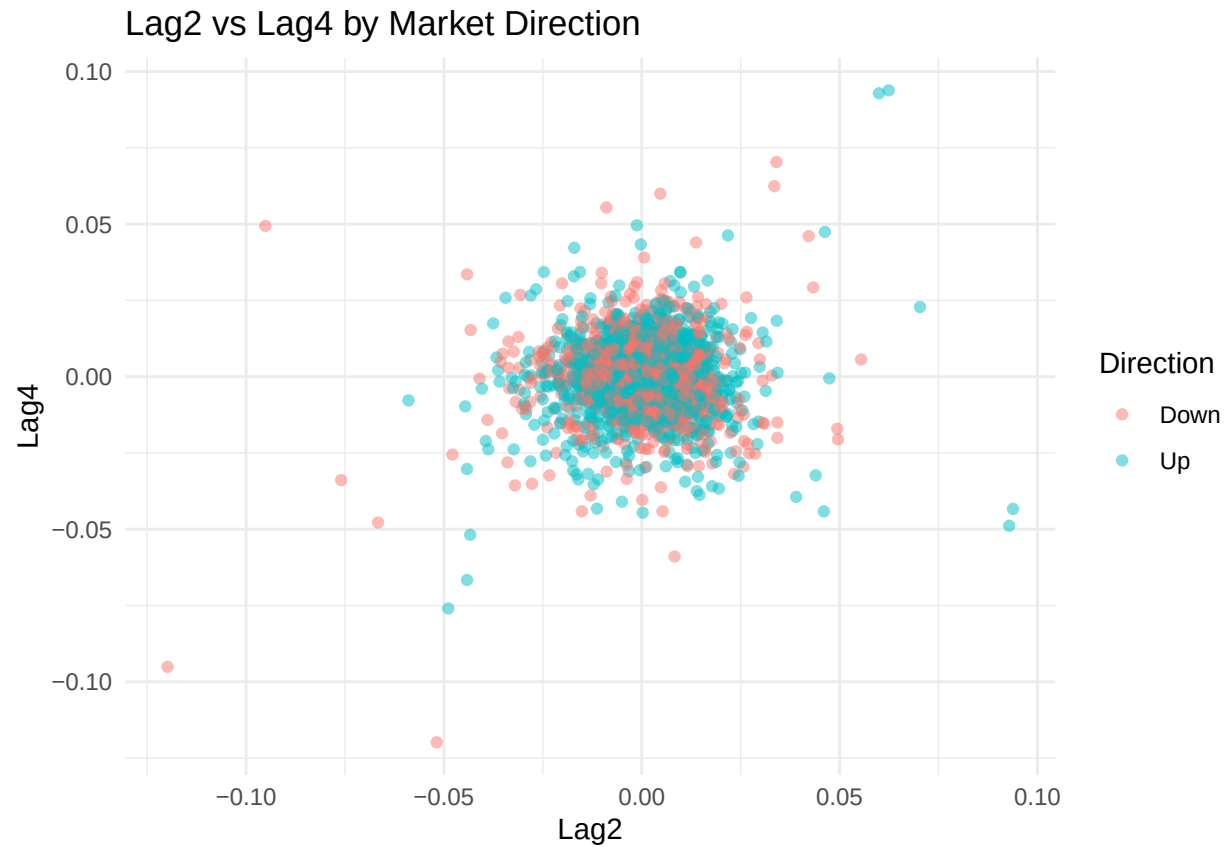
```
# Basic Scatter Plot: Lag 1 vs. Lag 2
ggplot(data, aes(x = Lag1, y = Lag2, color = Direction)) +
  geom_point(alpha = 0.5) +
  labs(
    title = "Lag1 vs Lag2 by Market Direction",
    x = "Lag1",
    y = "Lag2"
  ) +
  theme_minimal()
```



```
# Different Combination Scatter Plots
ggplot(data, aes(x = Lag1, y = Lag3, color = Direction)) +
  geom_point(alpha = 0.5) +
  labs(
    title = "Lag1 vs Lag3 by Market Direction",
    x = "Lag1",
    y = "Lag3"
  ) +
  theme_minimal()
```

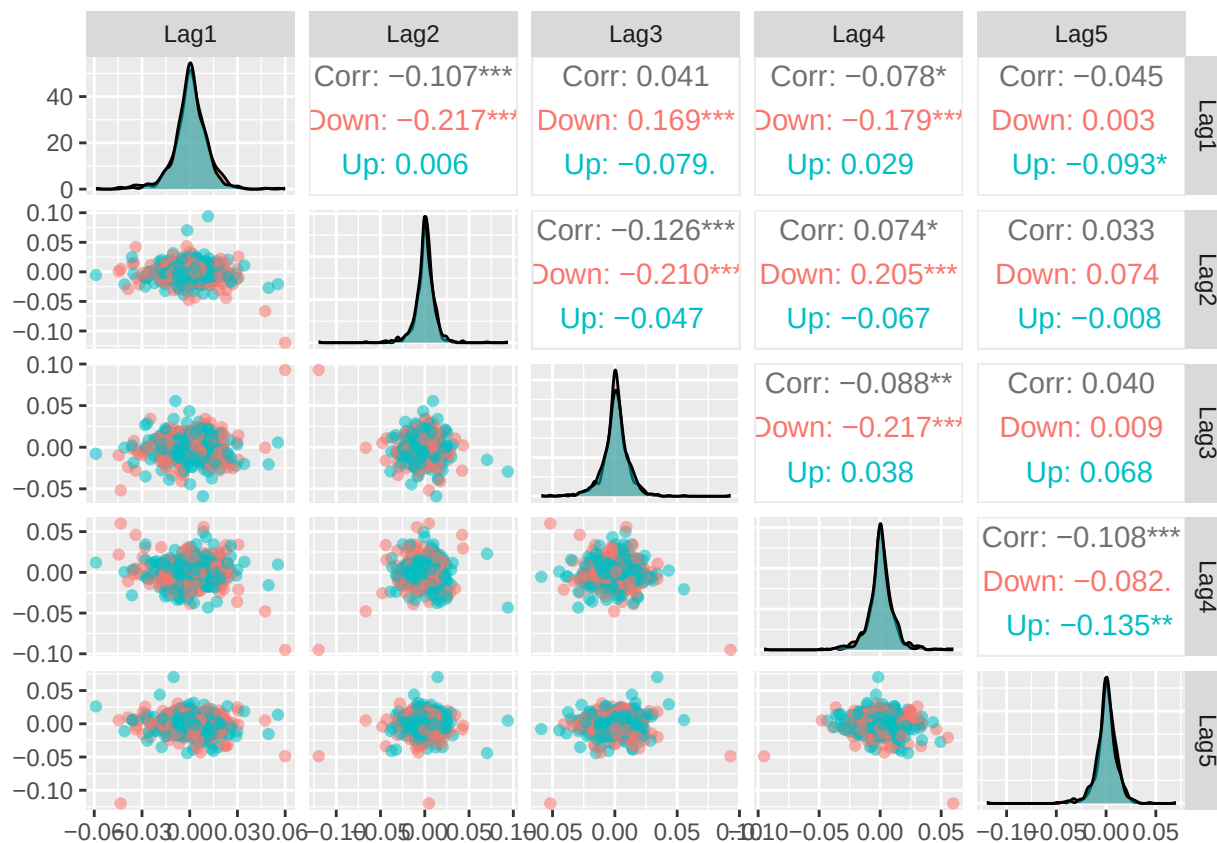


```
ggplot(data, aes(x = Lag2, y = Lag4, color = Direction)) +  
  geom_point(alpha = 0.5) +  
  labs(  
    title = "Lag2 vs Lag4 by Market Direction",  
    x = "Lag2",  
    y = "Lag4"  
  ) +  
  theme_minimal()
```

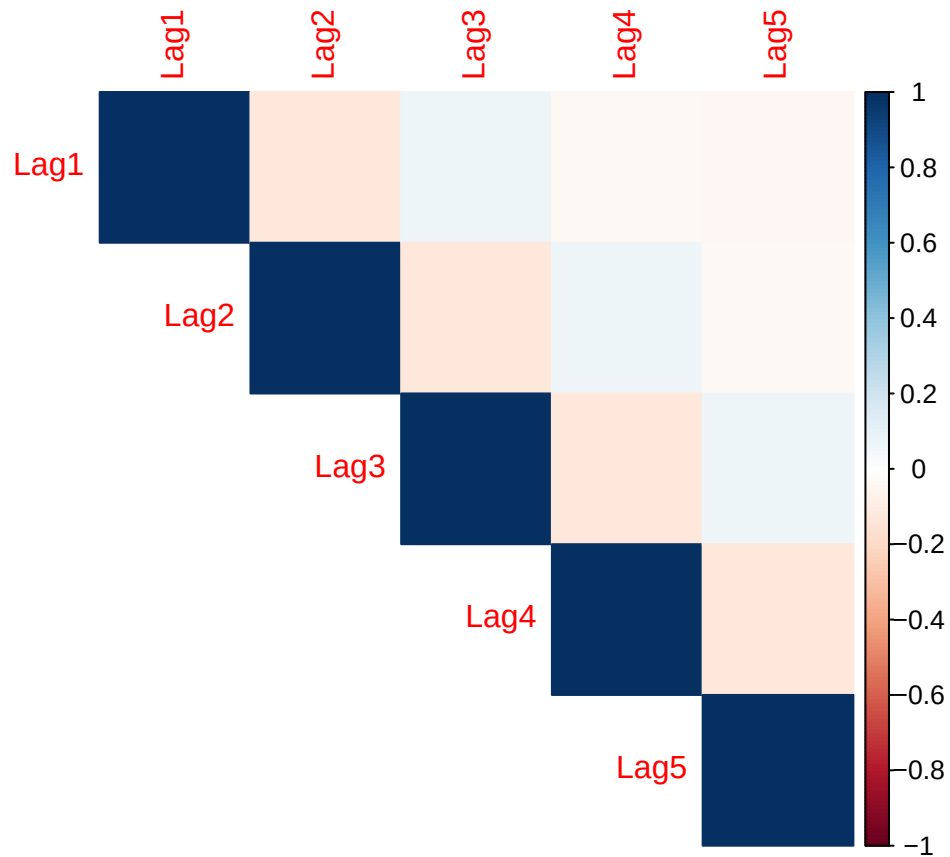


```
data_sample <- data[sample(nrow(data), 1000), ]

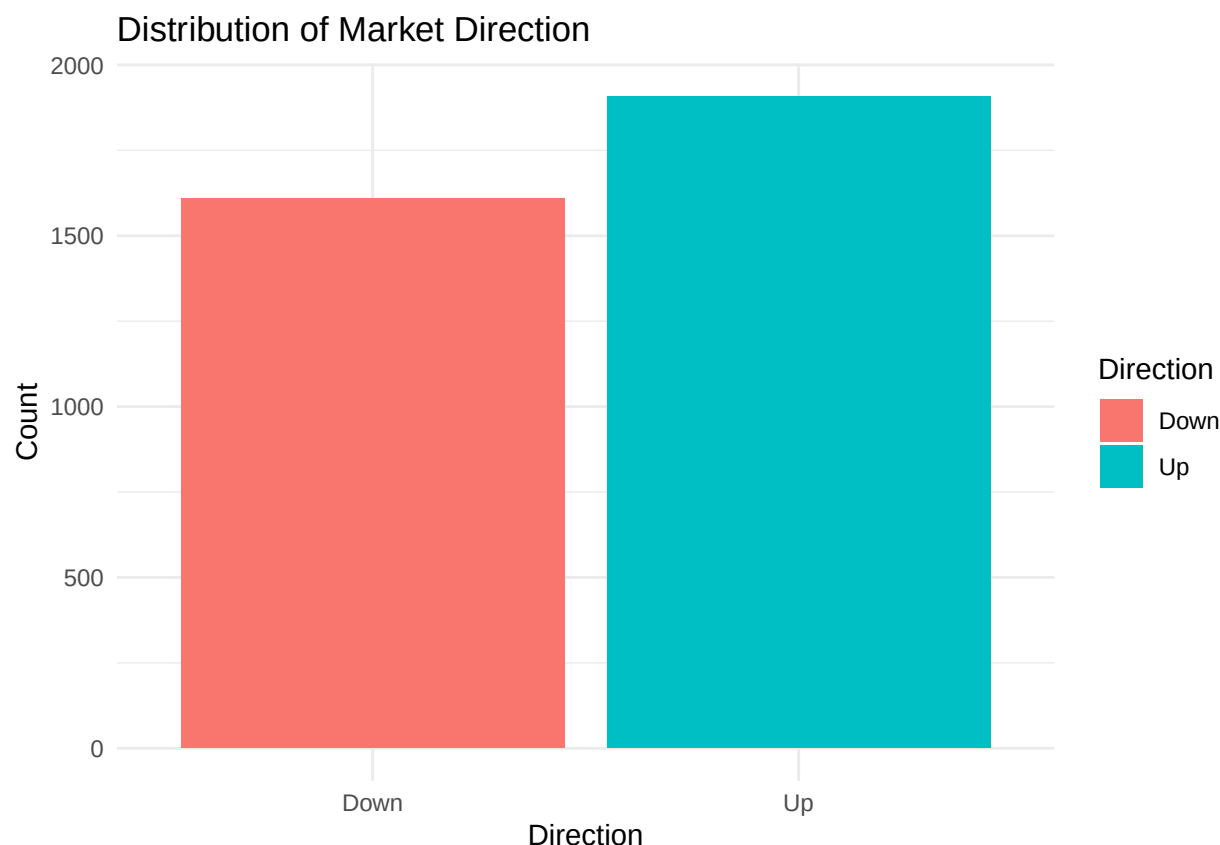
ggpairs(
  data_sample,
  columns = c("Lag1", "Lag2", "Lag3", "Lag4", "Lag5"),
  aes(color = Direction, alpha = 0.4)
)
```



```
corr_data <- cor(data[, c("Lag1", "Lag2", "Lag3", "Lag4", "Lag5")])
corrplot(corr_data, method = "color", type = "upper")
```

```
# Class Balance
ggplot(data, aes(x = Direction, fill = Direction)) +
  geom_bar() +
  labs(
    title = "Distribution of Market Direction",
    x = "Direction",
    y = "Count"
  ) +
  theme_minimal()
```



Interpretation:

The exploratory plots reveal substantial overlap between observations classified as “Up” and “Down”, with no clear separation between the two groups. Scatter plots such as Lag1 vs. Lag2 and Lag1 vs. Lag3 show that both classes occupy very similar regions in the feature space. The pair plot and correlation heatmap further confirm that lagged returns are only weakly correlated and do not exhibit strong distinguishing patterns. Overall, these results suggest that past returns provide limited information for predicting future market direction and that classification is inherently difficult.

One possible explanation is that daily returns are largely driven by new information arriving in the market, which is inherently unpredictable and not captured by lagged return variables alone.

5 Methodology

5.1 Training Strategy

The sample is split into a training set and a testing set based on time. Observations prior to 2022 are used for training, while observations from 2022 onward are used for testing. This preserves the time ordering of the data and better reflects a realistic forecasting setup.

```
# Train/Test Split
# Use data before 2022 as training, 2022 onward as testing
train_data <- data %>% filter(Date < as.Date("2022-01-01"))
test_data  <- data %>% filter(Date >= as.Date("2022-01-01"))

nrow(train_data)
```

```
## [1] 3016
nrow(test_data)
```

```
## [1] 501
```

5.2 Logistic Regression

```
logit_fit <- glm(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5,
                 data = train_data,
                 family = binomial)

summary(logit_fit)

##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5, family = binomial,
##      data = train_data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   0.21769    0.03707   5.872 4.3e-09 ***
## Lag1        -11.35830    3.56476  -3.186 0.00144 **
## Lag2           3.14820    3.52958   0.892 0.37242
## Lag3          -3.20448    3.57571  -0.896 0.37016
## Lag4          -1.28452    3.53798  -0.363 0.71656
## Lag5          -5.91721    3.53645  -1.673 0.09429 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 4149.1  on 3015  degrees of freedom
## Residual deviance: 4132.8  on 3010  degrees of freedom
## AIC: 4144.8
##
## Number of Fisher Scoring iterations: 4

logit_probs <- predict(logit_fit, newdata = test_data, type = "response")
logit_pred <- ifelse(logit_probs > 0.5, "Up", "Down")
logit_pred <- factor(logit_pred, levels = c("Down", "Up"))

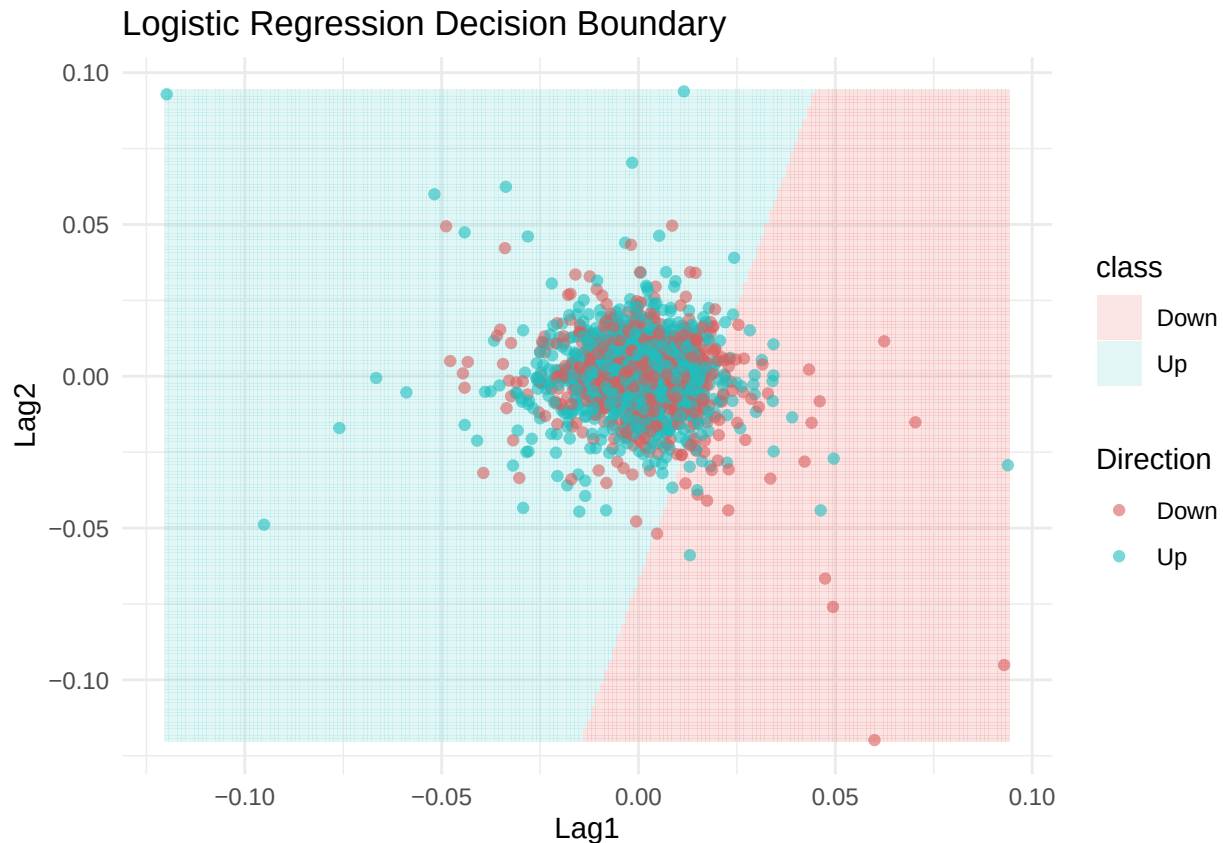
lag1_seq <- seq(min(train_data$Lag1), max(train_data$Lag1), length.out = 200)
lag2_seq <- seq(min(train_data$Lag2), max(train_data$Lag2), length.out = 200)

grid <- expand.grid(Lag1 = lag1_seq, Lag2 = lag2_seq)
grid$Lag3 <- mean(train_data$Lag3)
grid$Lag4 <- mean(train_data$Lag4)
grid$Lag5 <- mean(train_data$Lag5)

grid$prob <- predict(logit_fit, newdata = grid, type = "response")
grid$class <- ifelse(grid$prob > 0.5, "Up", "Down")
grid$class <- factor(grid$class, levels = c("Down", "Up"))

ggplot() +
```

```
geom_tile(data = grid, aes(x = Lag1, y = Lag2, fill = class), alpha = 0.3) +
geom_point(data = train_data, aes(x = Lag1, y = Lag2, color = Direction), alpha = 0.6) +
labs(
  title = "Logistic Regression Decision Boundary",
  x = "Lag1",
  y = "Lag2"
) +
theme_minimal() +
scale_fill_manual(values = c("Down" = "#F4A6A6", "Up" = "#9FE2E2")) +
scale_color_manual(values = c("Down" = "#D95F5F", "Up" = "#1FBFBF"))
```



Interpretation:

The logistic regression results indicate that most lagged variables are not statistically significant predictors of market direction. At most, Lag1 shows limited evidence of predictive value, but the overall model fit remains weak, indicating that lagged returns do not provide strong predictive power for classifying market direction.

5.3 LDA

```
lda_fit <- lda(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5,
  data = train_data)
```

```
lda_fit
```

```
## Call:
```

```

## lda(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5, data = train_data)
##
## Prior probabilities of groups:
##      Down      Up
## 0.4486074 0.5513926
##
## Group means:
##      Lag1      Lag2      Lag3      Lag4      Lag5
## Down 1.264965e-03 0.0002001652 0.0008398541 0.0004877235 0.0008776246
## Up -6.137519e-05 0.0008091391 0.0002881798 0.0005771506 0.0002516093
##
## Coefficients of linear discriminants:
##      LD1
## Lag1 -74.92413
## Lag2 20.68676
## Lag3 -19.61654
## Lag4 -8.46331
## Lag5 -38.96608

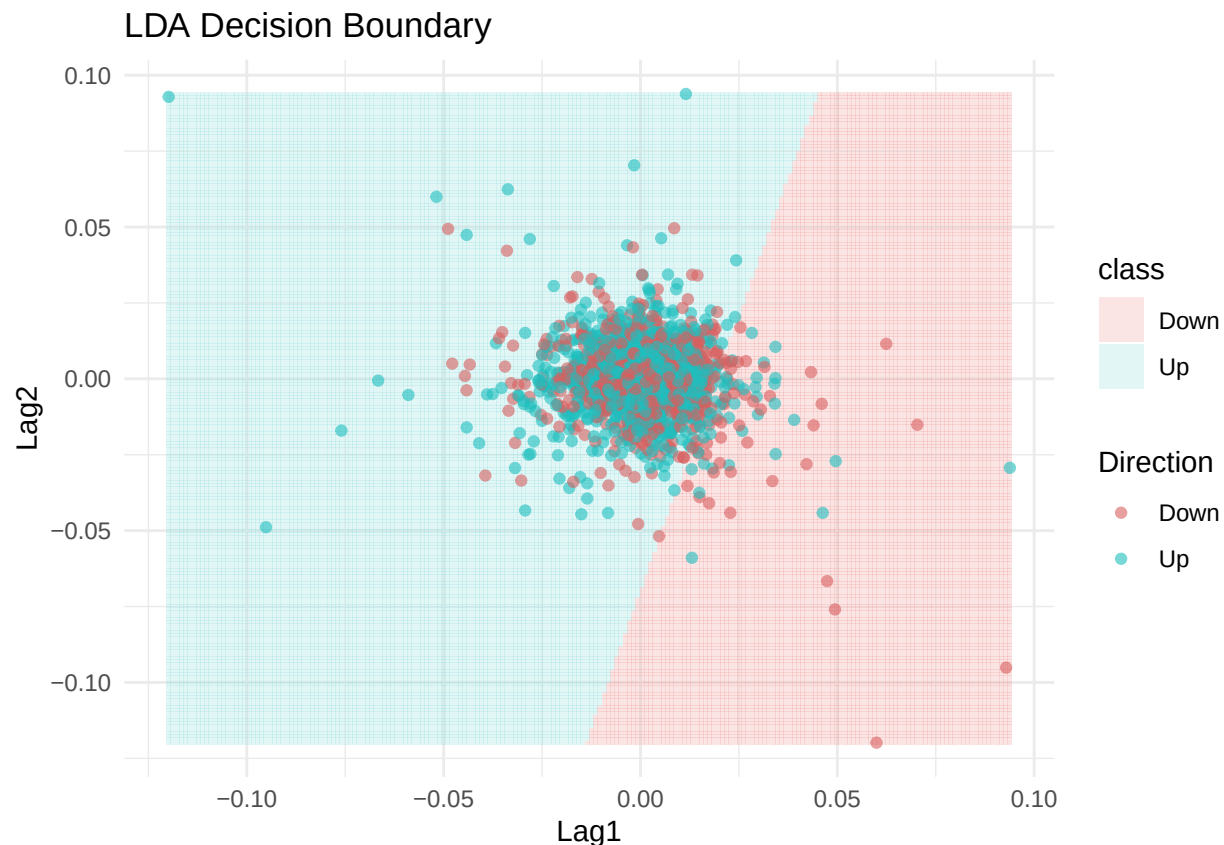
lda_pred <- predict(lda_fit, newdata = test_data)$class

grid <- expand.grid(Lag1 = lag1_seq, Lag2 = lag2_seq)
grid$Lag3 <- mean(train_data$Lag3)
grid$Lag4 <- mean(train_data$Lag4)
grid$Lag5 <- mean(train_data$Lag5)

lda_grid_pred <- predict(lda_fit, newdata = grid)
grid$class <- lda_grid_pred$class
grid$prob <- lda_grid_pred$posterior[, "Up"]

ggplot() +
  geom_tile(data = grid, aes(x = Lag1, y = Lag2, fill = class), alpha = 0.3) +
  geom_point(data = train_data, aes(x = Lag1, y = Lag2, color = Direction), alpha = 0.6) +
  labs(
    title = "LDA Decision Boundary",
    x = "Lag1",
    y = "Lag2"
  ) +
  theme_minimal() +
  scale_fill_manual(values = c("Down" = "#F4A6A6", "Up" = "#9FE2E2")) +
  scale_color_manual(values = c("Down" = "#D95F5F", "Up" = "#1FBFBF"))

```



Interpretation:

LDA produces results similar to logistic regression, with limited separation between the two classes. The group means for the lagged variables are very close, indicating that these predictors do not strongly differentiate between upward and downward movements. This suggests that the linear decision boundary imposed by LDA is insufficient for capturing meaningful patterns in the data.

5.4 QDA

```
qda_fit <- qda(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5,
               data = train_data)
```

```
qda_fit
```

```
## Call:
## qda(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5, data = train_data)
##
## Prior probabilities of groups:
##      Down      Up
## 0.4486074 0.5513926
##
## Group means:
##      Lag1      Lag2      Lag3      Lag4      Lag5
## Down 1.264965e-03 0.0002001652 0.0008398541 0.0004877235 0.0008776246
## Up   -6.137519e-05 0.0008091391 0.0002881798 0.0005771506 0.0002516093
```

```

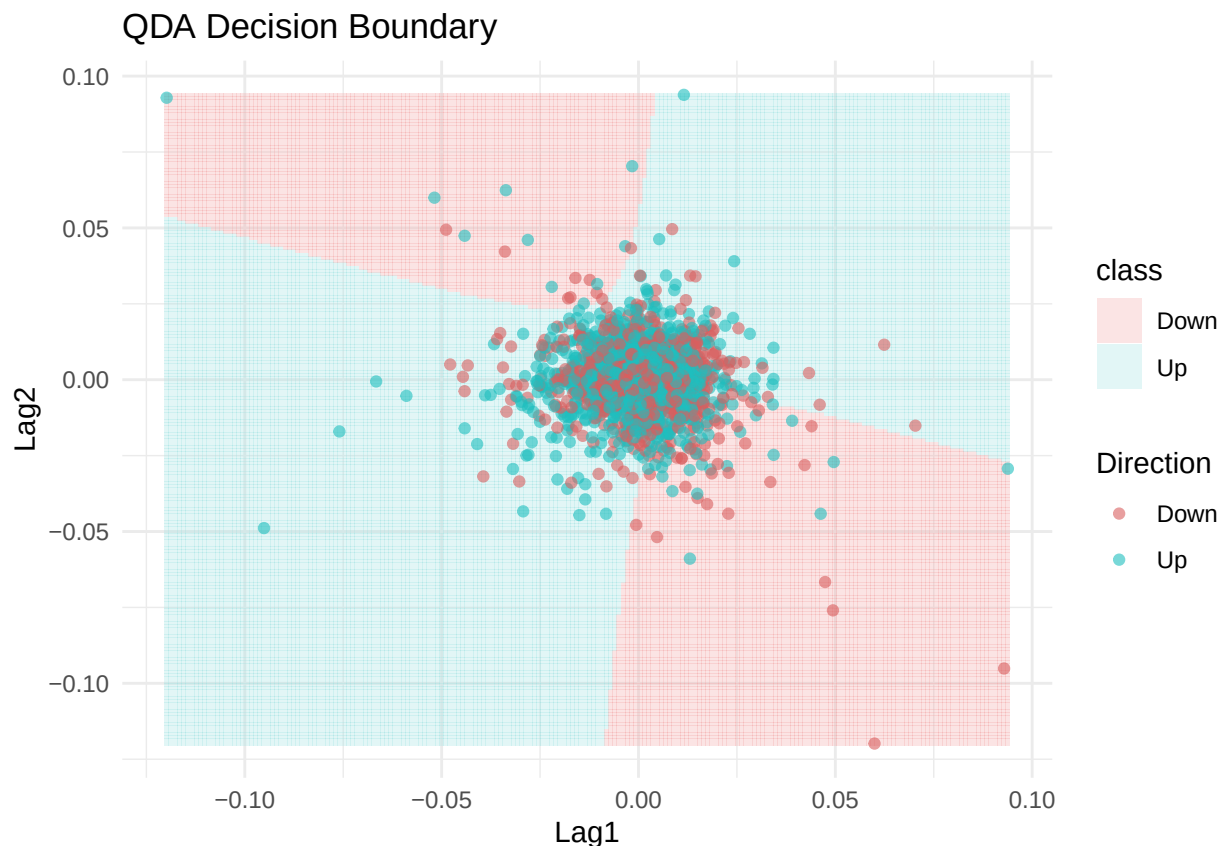
qda_pred <- predict(qda_fit, newdata = test_data)$class
qda_probs <- predict(qda_fit, newdata = test_data)$posterior[, "Up"]

grid <- expand.grid(Lag1 = lag1_seq, Lag2 = lag2_seq)
grid$Lag3 <- mean(train_data$Lag3)
grid$Lag4 <- mean(train_data$Lag4)
grid$Lag5 <- mean(train_data$Lag5)

qda_grid_pred <- predict(qda_fit, newdata = grid)
grid$class <- qda_grid_pred$class
grid$prob <- qda_grid_pred$posterior[, "Up"]

ggplot() +
  geom_tile(data = grid, aes(x = Lag1, y = Lag2, fill = class), alpha = 0.3) +
  geom_point(data = train_data, aes(x = Lag1, y = Lag2, color = Direction), alpha = 0.6) +
  labs(
    title = "QDA Decision Boundary",
    x = "Lag1",
    y = "Lag2"
  ) +
  theme_minimal() +
  scale_fill_manual(values = c("Down" = "#F4A6A6", "Up" = "#9FE2E2")) +
  scale_color_manual(values = c("Down" = "#D95F5F", "Up" = "#1FBFBF"))

```



Interpretation:

QDA allows for more flexible decision boundaries by modeling different covariance structures for each class. However, its performance is worse than both logistic regression and LDA. This suggests that increasing model flexibility does not improve predictive performance and may instead amplify noise when the underlying signal in the data is weak.

5.5 kNN

```
train_x <- train_data %>% dplyr::select(Lag1, Lag2, Lag3, Lag4, Lag5)
test_x  <- test_data  %>% dplyr::select(Lag1, Lag2, Lag3, Lag4, Lag5)

train_means <- sapply(train_x, mean)
train_sds   <- sapply(train_x, sd)

train_x_scaled <- scale(train_x, center = train_means, scale = train_sds)
test_x_scaled  <- scale(test_x, center = train_means, scale = train_sds)

train_y <- train_data$Direction
test_y  <- test_data$Direction

knn_pred <- knn(train = train_x_scaled,
                test = test_x_scaled,
                cl = train_y,
                k = 5)

knn_acc <- mean(knn_pred == test_y)
knn_acc

## [1] 0.5309381

grid <- expand.grid(Lag1 = lag1_seq, Lag2 = lag2_seq)
grid$Lag3 <- mean(train_data$Lag3)
grid$Lag4 <- mean(train_data$Lag4)
grid$Lag5 <- mean(train_data$Lag5)

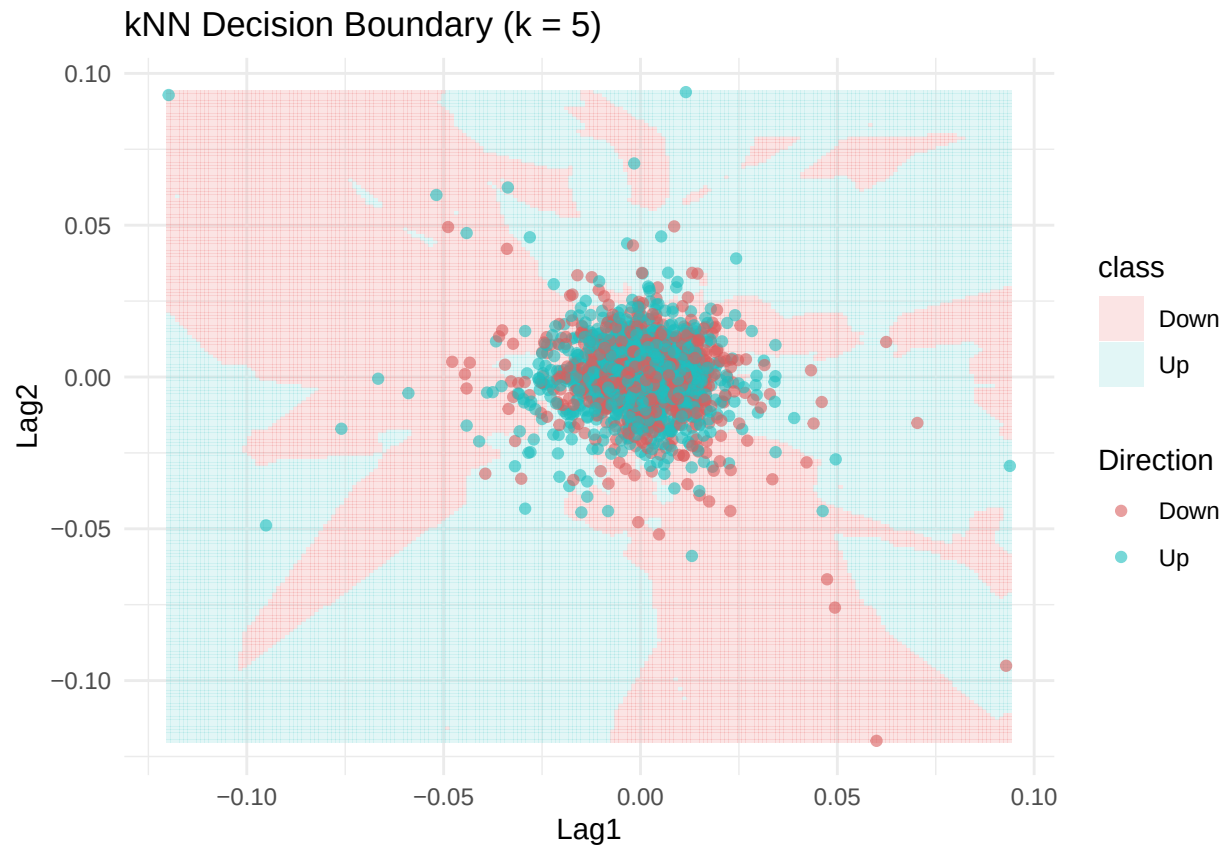
grid_scaled <- scale(
  grid[, c("Lag1", "Lag2", "Lag3", "Lag4", "Lag5")],
  center = train_means,
  scale = train_sds
)

grid$class <- knn(
  train = train_x_scaled,
  test = grid_scaled,
  cl = train_y,
  k = 5
)

ggplot() +
  geom_tile(data = grid, aes(x = Lag1, y = Lag2, fill = class), alpha = 0.3) +
  geom_point(data = train_data, aes(x = Lag1, y = Lag2, color = Direction), alpha = 0.6) +
  labs(
    title = "kNN Decision Boundary (k = 5)",
    x = "Lag1",
    y = "Lag2"
  )
```



```
) +
theme_minimal() +
scale_fill_manual(values = c("Down" = "#F4A6A6", "Up" = "#9FE2E2")) +
scale_color_manual(values = c("Down" = "#D95F5F", "Up" = "#1FBFBF"))
```



Interpretation:

The kNN model achieves the highest test accuracy among the models considered. However, the improvement is modest, with accuracy only slightly above 50%. This suggests that while local nonlinear patterns may exist in the data, they are weak and do not substantially enhance predictive performance.

5.6 Cross Validation for ALL Models

To ensure a fair comparison across methods, 10-fold cross-validation is implemented not only for kNN but also for logistic regression, LDA, and QDA. This provides a more rigorous evaluation of model performance and helps determine whether the conclusions hold across repeated resampling.

```
ctrl <- trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)

# caret expects the first level to be the "event" for twoClassSummary
train_data_cv <- train_data
```

```

train_data_cv$Direction <- factor(train_data_cv$Direction, levels = c("Up", "Down"))

logit_cv <- train(
  Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5,
  data = train_data_cv,
  method = "glm",
  family = "binomial",
  metric = "ROC",
  trControl = ctrl
)

lda_cv <- train(
  Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5,
  data = train_data_cv,
  method = "lda",
  metric = "ROC",
  trControl = ctrl
)

qda_cv <- train(
  Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5,
  data = train_data_cv,
  method = "qda",
  metric = "ROC",
  trControl = ctrl
)

knn_cv <- train(
  Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5,
  data = train_data_cv,
  method = "knn",
  metric = "ROC",
  trControl = ctrl,
  tuneLength = 15,
  preProcess = c("center", "scale")
)

logit_cv

```

```

## Generalized Linear Model
##
## 3016 samples
##    5 predictor
##    2 classes: 'Up', 'Down'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 2715, 2714, 2714, 2715, 2713, 2714, ...
## Resampling results:
##
##      ROC          Sens          Spec
## 0.5282014 0.9609372 0.04805011

```

```
lda_cv
```

```
## Linear Discriminant Analysis
##
## 3016 samples
##    5 predictor
##    2 classes: 'Up', 'Down'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 2713, 2715, 2715, 2715, 2714, 2715, ...
## Resampling results:
##
##      ROC          Sens          Spec
## 0.5326642 0.9632927 0.04435185
```

```
qda_cv
```

```
## Quadratic Discriminant Analysis
##
## 3016 samples
##    5 predictor
##    2 classes: 'Up', 'Down'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 2714, 2715, 2715, 2713, 2715, 2714, ...
## Resampling results:
##
##      ROC          Sens          Spec
## 0.5087078 0.8682599 0.1448475
```

```
knn_cv
```

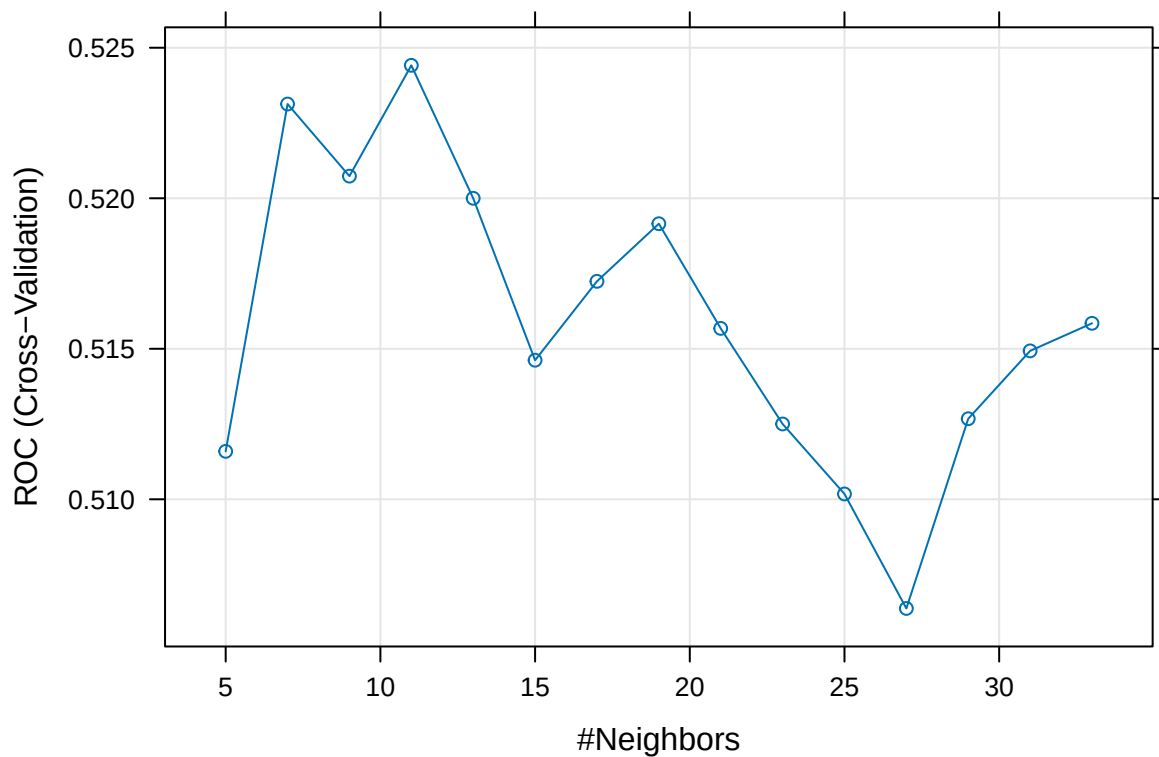
```
## k-Nearest Neighbors
##
## 3016 samples
##    5 predictor
##    2 classes: 'Up', 'Down'
##
## Pre-processing: centered (5), scaled (5)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 2714, 2715, 2714, 2715, 2714, 2715, ...
## Resampling results across tuning parameters:
##
##    k   ROC          Sens          Spec
##    5 0.5115928 0.6019154 0.4257135
##    7 0.5231300 0.6313794 0.4197930
##    9 0.5207344 0.6464180 0.4035948
##   11 0.5244159 0.6500216 0.3999237
##   13 0.5199987 0.6518361 0.3858987
##   15 0.5146202 0.6626722 0.3711002
##   17 0.5172401 0.6722819 0.3511111
##   19 0.5191549 0.6891133 0.3370534
##   21 0.5156768 0.6897121 0.3378159
```

```
## 23 0.5125016 0.6951122 0.3252179
## 25 0.5101775 0.6969230 0.3267266
## 27 0.5063739 0.6945062 0.3067647
## 29 0.5126754 0.7065399 0.3237364
## 31 0.5149304 0.7161496 0.3059913
## 33 0.5158462 0.7185737 0.3023312
##
## ROC was used to select the optimal model using the largest value.
## The final value used for the model was k = 11.
```

```
best_k <- knn_cv$bestTune$k
```

```
knn_best_pred <- knn(
  train = train_x_scaled,
  test = test_x_scaled,
  cl = train_y,
  k = best_k
)
```

```
plot(knn_cv)
```



Interpretation:

The cross-validation results show that all models achieve ROC values close to 0.5, indicating very weak predictive performance. Logistic regression and LDA perform slightly better than QDA, while kNN achieves comparable performance after tuning. Although cross-validation helps select the optimal hyperparameters and reduces overfitting, it does not lead to meaningful improvements in prediction accuracy. This suggests

that the limitation lies in the data itself rather than the modeling approach, as lagged returns contain little useful information for forecasting market direction.

6 Results

6.1 Accuracy Comparison

```
logit_acc <- mean(logit_pred == test_data$Direction)
lda_acc   <- mean(lda_pred == test_data$Direction)
qda_acc   <- mean(qda_pred == test_data$Direction)
knn_acc   <- mean(knn_pred == test_y)
knn_best_acc <- mean(knn_best_pred == test_y)

cv_results <- data.frame(
  Model = c("Logistic Regression", "LDA", "QDA", "kNN"),
  Test_Accuracy = c(logit_acc, lda_acc, qda_acc, knn_best_acc),
  CV_ROC = c(
    max(logit_cv$results$ROC),
    max(lda_cv$results$ROC),
    max(qda_cv$results$ROC),
    max(knn_cv$results$ROC)
  ),
  Type = c("Linear", "Linear", "Nonlinear", "Nonlinear")
)

cv_results
```

##	Model	Test_Accuracy	CV_ROC	Type
## 1	Logistic Regression	0.4990020	0.5282014	Linear
## 2	LDA	0.4950100	0.5326642	Linear
## 3	QDA	0.4770459	0.5087078	Nonlinear
## 4	kNN	0.5269461	0.5244159	Nonlinear

Interpretation:

The accuracy results show that all models perform close to 50%, indicating that they are only slightly better than random guessing. While kNN achieves the highest test accuracy, the improvement over the linear models is minimal. The cross-validation ROC values, all around 0.51–0.53, further confirm that none of the models have strong predictive ability. Overall, these results suggest that both linear and nonlinear models struggle to extract meaningful signals from lagged returns, reinforcing the conclusion that predictive power is very limited.

6.2 Training vs Testing Accuracy

```
logit_train_probs <- predict(logit_fit, type = "response")
logit_train_pred <- ifelse(logit_train_probs > 0.5, "Up", "Down")
logit_train_pred <- factor(logit_train_pred, levels = c("Down", "Up"))

lda_train_acc <- mean(predict(lda_fit)$class == train_data$Direction)
qda_train_acc <- mean(predict(qda_fit)$class == train_data$Direction)
logit_train_acc <- mean(logit_train_pred == train_data$Direction)
knn_train_pred <- knn(train_x_scaled, train_x_scaled, train_y, k = 5)
```

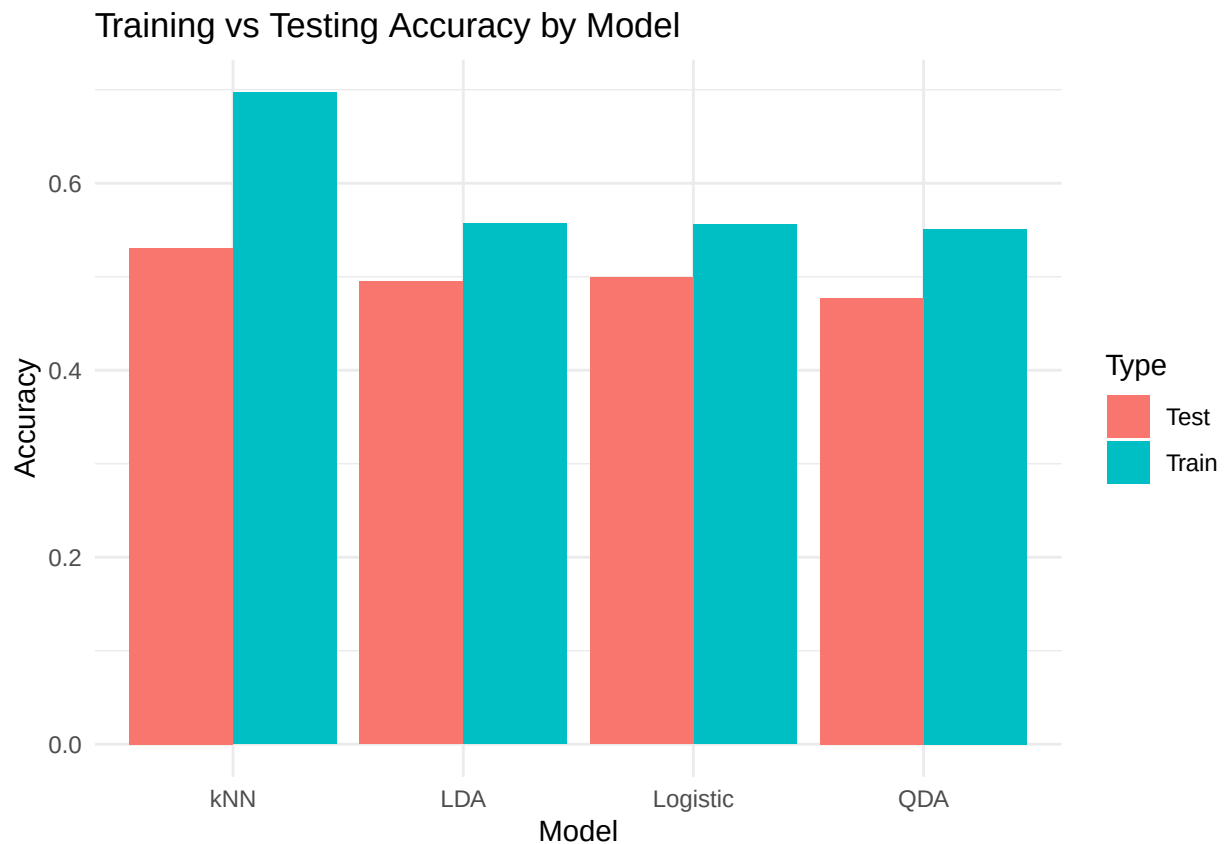
```

knn_train_acc <- mean(knn_train_pred == train_y)

performance <- data.frame(
  Model = rep(c("Logistic", "LDA", "QDA", "kNN"), each = 2),
  Type = rep(c("Train", "Test"), times = 4),
  Accuracy = c(
    logit_train_acc, logit_acc,
    lda_train_acc, lda_acc,
    qda_train_acc, qda_acc,
    knn_train_acc, knn_acc
  )
)

ggplot(performance, aes(x = Model, y = Accuracy, fill = Type)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Training vs Testing Accuracy by Model") +
  theme_minimal()

```



Interpretation:

Training accuracy is consistently higher than testing accuracy across all models, indicating some degree of overfitting. This is particularly evident for kNN, which achieves higher training accuracy due to its flexibility. However, the gap between training and testing performance is relatively small, suggesting that none of the models severely overfit the data.

6.3 Confusion Matrices

```
confusionMatrix(logit_pred, test_data$Direction)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Down  Up
##           Down   20  15
##           Up    236 230
##
##           Accuracy : 0.499
##           95% CI : (0.4543, 0.5437)
##           No Information Rate : 0.511
##           P-Value [Acc > NIR] : 0.7194
##
##           Kappa : 0.0166
##
## Mcnemar's Test P-Value : <2e-16
##
##           Sensitivity : 0.07812
##           Specificity : 0.93878
##           Pos Pred Value : 0.57143
##           Neg Pred Value : 0.49356
##           Prevalence : 0.51098
##           Detection Rate : 0.03992
##           Detection Prevalence : 0.06986
##           Balanced Accuracy : 0.50845
##
##           'Positive' Class : Down
##
```

```
confusionMatrix(lda_pred, test_data$Direction)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Down  Up
##           Down   18  15
##           Up    238 230
##
##           Accuracy : 0.495
##           95% CI : (0.4504, 0.5397)
##           No Information Rate : 0.511
##           P-Value [Acc > NIR] : 0.7763
##
##           Kappa : 0.0089
##
## Mcnemar's Test P-Value : <2e-16
##
##           Sensitivity : 0.07031
##           Specificity : 0.93878
##           Pos Pred Value : 0.54545
##           Neg Pred Value : 0.49145
##           Prevalence : 0.51098
```

```
##          Detection Rate : 0.03593
##    Detection Prevalence : 0.06587
##      Balanced Accuracy : 0.50454
##
##      'Positive' Class : Down
##
confusionMatrix(qda_pred, test_data$Direction)
```

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction Down  Up
##      Down   45  51
##      Up    211 194
##
##          Accuracy : 0.477
##          95% CI : (0.4326, 0.5218)
##    No Information Rate : 0.511
##    P-Value [Acc > NIR] : 0.9411
##
##          Kappa : -0.0319
##
## Mcnemar's Test P-Value : <2e-16
##
##          Sensitivity : 0.17578
##          Specificity : 0.79184
##      Pos Pred Value : 0.46875
##      Neg Pred Value : 0.47901
##          Prevalence : 0.51098
##      Detection Rate : 0.08982
##    Detection Prevalence : 0.19162
##      Balanced Accuracy : 0.48381
##
##      'Positive' Class : Down
##
```

```
confusionMatrix(knn_pred, test_y)
```

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction Down  Up
##      Down   116  95
##      Up    140 150
##
##          Accuracy : 0.5309
##          95% CI : (0.4862, 0.5753)
##    No Information Rate : 0.511
##    P-Value [Acc > NIR] : 0.197960
##
##          Kappa : 0.0651
##
## Mcnemar's Test P-Value : 0.004102
##
```



```
##          Sensitivity : 0.4531
##          Specificity : 0.6122
##          Pos Pred Value : 0.5498
##          Neg Pred Value : 0.5172
##          Prevalence : 0.5110
##          Detection Rate : 0.2315
##          Detection Prevalence : 0.4212
##          Balanced Accuracy : 0.5327
##
##          'Positive' Class : Down
##
```

```
confusionMatrix(knn_best_pred, test_y)
```

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction Down Up
##          Down 105 86
##          Up   151 159
##
##          Accuracy : 0.5269
##          95% CI : (0.4822, 0.5714)
##          No Information Rate : 0.511
##          P-Value [Acc > NIR] : 0.2514
##
##          Kappa : 0.0588
##
##          Mcnemar's Test P-Value : 3.221e-05
##
##          Sensitivity : 0.4102
##          Specificity : 0.6490
##          Pos Pred Value : 0.5497
##          Neg Pred Value : 0.5129
##          Prevalence : 0.5110
##          Detection Rate : 0.2096
##          Detection Prevalence : 0.3812
##          Balanced Accuracy : 0.5296
##
##          'Positive' Class : Down
##
```

Interpretation:

The confusion matrices reveal that all models struggle to correctly classify both upward and downward market movements. Logistic regression and LDA exhibit very low sensitivity, meaning they fail to correctly identify many downward movements. QDA performs worse overall, reflecting its tendency to overfit noise. While kNN provides a more balanced classification and slightly higher accuracy, it still produces a large number of misclassifications. Overall, these results confirm that predictive performance is limited and that no model achieves reliable classification accuracy.

6.4 ROC Curves and AUC

To complement accuracy, ROC curves and AUC are used to evaluate each model's ability to discriminate between the two classes across all classification thresholds.

```

# Relevel so that "Up" is the positive class for ROC calculations
test_direction_up <- factor(test_data$Direction, levels = c("Down", "Up"))

lda_probs <- predict(lda_fit, newdata = test_data)$posterior[, "Up"]

# QDA probabilities were created earlier
# kNN class::knn does not return probabilities, so use caret probabilities
knn_cv_probs <- predict(knn_cv, newdata = test_data, type = "prob")[, "Up"]

roc_logit <- roc(response = test_direction_up, predictor = logit_probs, levels = c("Down", "Up"), direction = "up")
roc_lda <- roc(response = test_direction_up, predictor = lda_probs, levels = c("Down", "Up"), direction = "up")
roc_qda <- roc(response = test_direction_up, predictor = qda_probs, levels = c("Down", "Up"), direction = "up")
roc_knn <- roc(response = test_direction_up, predictor = knn_cv_probs, levels = c("Down", "Up"), direction = "up")

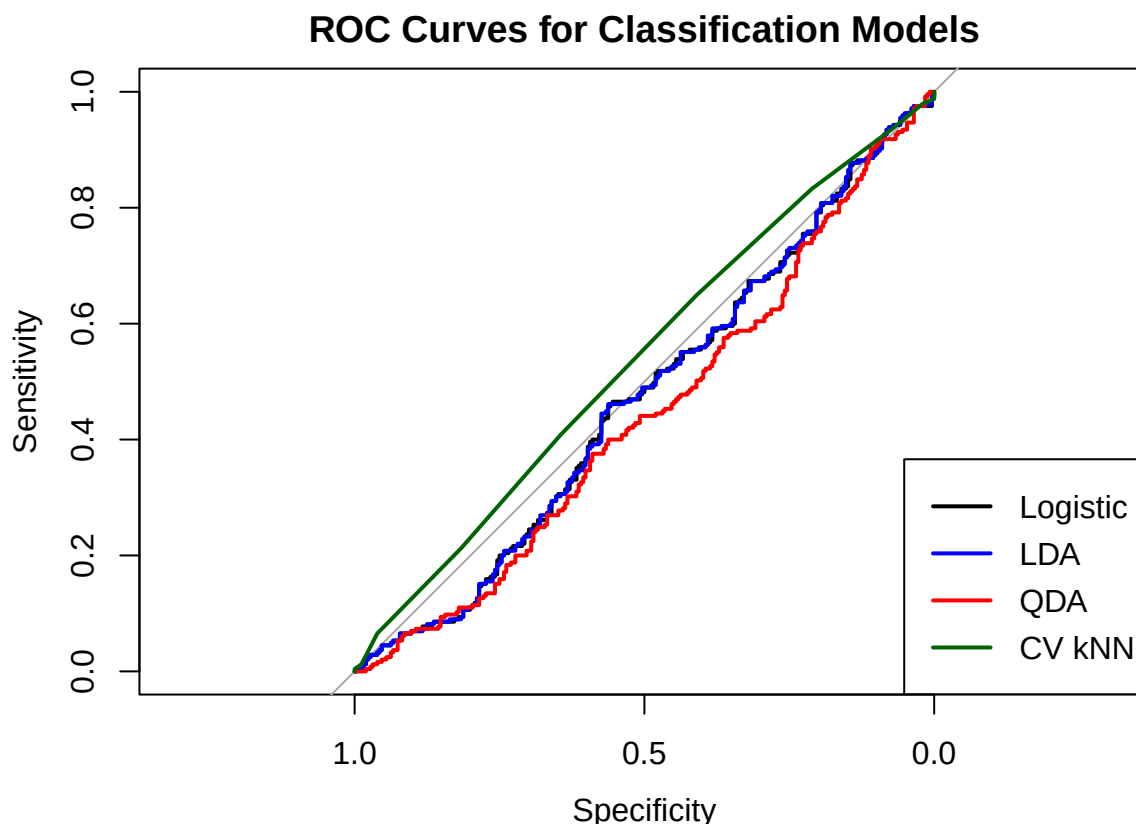
auc_table <- data.frame(
  Model = c("Logistic Regression", "LDA", "QDA", "CV kNN"),
  AUC = c(
    as.numeric(auc(roc_logit)),
    as.numeric(auc(roc_lda)),
    as.numeric(auc(roc_qda)),
    as.numeric(auc(roc_knn))
  )
)

auc_table

##           Model      AUC
## 1 Logistic Regression 0.4710619
## 2           LDA 0.4713489
## 3           QDA 0.4430325
## 4          CV kNN 0.5383689

plot(roc_logit, col = "black", main = "ROC Curves for Classification Models")
plot(roc_lda, add = TRUE, col = "blue")
plot(roc_qda, add = TRUE, col = "red")
plot(roc_knn, add = TRUE, col = "darkgreen")
legend(
  "bottomright",
  legend = c("Logistic", "LDA", "QDA", "CV kNN"),
  col = c("black", "blue", "red", "darkgreen"),
  lwd = 2
)

```



Interpretation:

The ROC curves further confirm the limited predictive power of all models. The AUC values are close to 0.5 for logistic regression, LDA, and QDA, indicating that these models perform no better than random guessing. The cross-validated kNN model achieves the highest AUC (approximately 0.54), but this improvement is still very small and not practically meaningful. Overall, the ROC analysis reinforces the conclusion that lagged returns provide little useful information for predicting market direction.

6.5 Linear vs. Nonlinear Discussion

```
linear_acc <- mean(c(logit_acc, lda_acc))
nonlinear_acc <- mean(c(qda_acc, knn_best_acc))

comparison <- data.frame(
  Type = c("Linear Models", "Nonlinear Models"),
  AverageAccuracy = c(linear_acc, nonlinear_acc)
)
```

```
comparison
```

```
##           Type AverageAccuracy
## 1  Linear Models      0.497006
## 2 Nonlinear Models    0.501996
```

Interpretation:

Although nonlinear models show a slight improvement in accuracy compared to linear models, the difference is extremely small and not statistically or economically meaningful. This suggests that the underlying

relationship between lagged returns and market direction is either very weak or nonexistent. Increasing model flexibility does not uncover additional structure in the data, indicating that the limitation lies in the predictors rather than the model choice.

7 Conclusion and Future Work

7.1 Economic Interpretation

From an economic perspective, the results of this study are consistent with the weak-form Efficient Market Hypothesis. The low predictive accuracy observed across all models suggests that historical returns contain little exploitable information for forecasting future market direction. While kNN achieves slightly higher accuracy than the other methods, the gain is minimal and unlikely to be economically meaningful.

From a practical standpoint, a predictive accuracy of approximately 53% is unlikely to generate consistent profits once transaction costs, bid-ask spreads, trading frictions, and risk are considered. In many financial settings, a model would need substantially higher predictive accuracy to be economically viable. Therefore, the observed performance is not only statistically weak but also economically insignificant.

7.2 Final Thoughts

Overall, this study demonstrates that stock market direction is difficult to predict using lagged financial indicators alone. While classification models provide a useful framework for evaluating predictive relationships, their performance remains close to random guessing in this setting. In addition, nonlinear models do not meaningfully outperform linear alternatives, suggesting that increasing model complexity does not substantially improve prediction.

Ultimately, this study highlights a fundamental challenge in financial forecasting: even reasonably sophisticated statistical models struggle to outperform randomness when relying solely on historical price information.

7.3 Future Work

Future research could explore the inclusion of additional predictors beyond lagged returns, such as macroeconomic variables, interest rates, volatility indices, trading volume, or investor sentiment measures. These variables may capture broader economic conditions that are not reflected in past returns alone.

In addition, more advanced machine learning methods, such as random forests, support vector machines, gradient boosting, or neural networks, could be applied to test whether more complex nonlinear relationships exist. Future analysis could also examine different forecasting horizons to determine whether predictability varies across time scales.

8 References

Dave's Monetary Economics. (n.d.). Trading costs and efficient financial markets. <https://sites.google.com/site/davesmant/monetary-economics/rational-efficient-financial-markets/trading-costs>

James, G., Witten, D., Hastie, T., & Tibshirani, R. (n.d.). Classification lab. In *An Introduction to Statistical Learning* (Python edition). <https://islp.readthedocs.io/en/latest/labs/Ch04-classification-lab.html>

James, G., Witten, D., Hastie, T., & Tibshirani, R. (n.d.). Smarket dataset. In *An Introduction to Statistical Learning* (Python edition). <https://islp.readthedocs.io/en/latest/datasets/Smarket.html>